

Q1. Walk me through REZO.AI in 60 seconds. What does the product do, who are the customers, and where did your work sit in the pipeline?

Answer1: REZO.AI is a DIY voice AI platform for Indian enterprises. Businesses like [BFSI / e-commerce / support-heavy companies] build their own phone voicebots on it without writing code: they configure the prompt, knowledge base, voice, and language. It supports on-prem deployment for customers who can't send call audio to the cloud, and the core engineering constraint is keeping end-to-end latency under 800ms, so the bot feels like a human on a call.

I was a product engineer intern on a 5-person team. My work touched two layers. On the **builder side**, I built the RAG test panel, so a builder can check retrieval quality before a bot goes live. On the **runtime pipeline**, I worked on three pieces around the LLM: a pronunciation layer just before TTS, backchannel handling in turn-taking so 'haan' or 'hmm' doesn't cut the bot off, and debugging a language-detection bug in STT.

The common theme was call quality: making the bot sound right, listen right, and answer from the right data.

Q2. Draw the voicebot pipeline end to end: **user speaks** → ... → **user hears the bot**. Name each stage, and give a rough split of the 800ms budget across them.

Answer2:

User speaks (phone / WebRTC)

→ Telephony / transport (SIP or WebRTC, 8kHz audio frames)

→ Denoiser (noise suppression)

→ VAD + endpointing (is the user speaking? have they finished their turn?)

→ STT (streaming) (audio → text; partial + final transcripts)

→ [RAG retrieval] (fetch KB chunks if needed)

→ LLM (streaming) (tokens out)

→ Sentence chunker (group tokens into speakable chunks)

→ Pre-TTS substitution ← YOUR pronunciation layer

→ TTS (streaming) (text → audio)

→ Transport back → user hears the bot

Bullet	Stage
Backchannel supervision	VAD / interruption handling
STT language detection bug	STT, plus how its output reaches the LLM prompt
RAG test panel	Builder tooling for the retrieval stage
Pronunciation dictionary	Between the chunker and TTS

800ms budget (rough, typical numbers):

Stage	~ms
Endpointing (silence wait to decide the user is done)	200–300
STT final transcript	100–150
LLM time-to-first-token	200–300
TTS time-to-first-byte	100–150
Network / transport	50–100

Q2a. What's the difference between VAD and endpointing? Why can't you just say "the user stopped when VAD says silence"?

Q2b. Give me 3 ways to cut latency if the bot is at 1.2s.

Q2c. What is barge-in, and how does the pipeline handle the user interrupting the bot mid-sentence?

Answer2(a): "People pause mid-sentence. They think, say 'umm', or read out a number digit by digit. If I end the turn on the first 300ms of silence, I cut them off. If I wait 1.5s to be safe, I've blown the latency budget. So endpointing is a trade-off between cutting users off and adding latency. Better systems add a semantic signal on top of VAD: they check whether the transcript looks like a complete thought, and they adapt the silence timeout to context, for example waiting longer when the bot has just asked for an account number."

Answer2(b): "First I'd **measure**: put a timestamp on every stage to find where the time actually goes. Then I'd fix that stage."

Then the fixes, grouped by stage:

1. **Endpointing**: use semantic turn detection so the silence timeout can safely drop.
2. **STT → LLM**: speculative execution. Start the LLM on the partial transcript and throw the result away if the final transcript differs.
3. **LLM**: use a smaller or faster model, shorten the system prompt, use prompt caching, and keep the first sentence short.
4. **LLM → TTS**: chunk aggressively. Send the first clause to TTS instead of waiting for a full sentence.
5. **RAG**: skip retrieval when the query doesn't need it, and run retrieval in parallel with the LLM where possible.
6. **Infra**: co-locate STT, LLM, and TTS in one region (or on-prem), keep persistent websockets to providers, and use warm connections.
7. **Perceived latency**: pre-synthesize common lines like the greeting, and play a filler such as "ek second..." while slow RAG runs.

Answer2(c): "Barge-in is the user interrupting while the bot is speaking. VAD keeps running on the user's audio even during bot playback. When it detects sustained speech, the pipeline sends an interruption signal (in Pipecat, an interruption frame propagates down the pipeline) that:

1. Stops TTS playback immediately and flushes the queued audio
2. Cancels in-flight LLM and TTS generation
3. Truncates the bot's last message in the conversation history to what was *actually spoken*, so the LLM knows what the user heard
4. Processes the new user turn

There are two classic failure modes. **Echo**: the bot's own voice leaks into the mic and triggers an interruption, which acoustic echo cancellation fixes. **Backchannels**: the user says 'haan' or 'hmm' just to show they're listening, VAD sees speech, and the bot cuts itself off. **That second one is exactly what I fixed.**"

Q3. "O(1) rule lookups." How do you match multi-word phrases like "Bajaj Allianz" in O(1)? What's actually O(1), and what isn't?

Answer: **What's O(1)**: looking up one candidate string in a hash map (dict keyed on the normalized term). That cost doesn't depend on dictionary size, whether you have 10 rules or 10,000.

What isn't O(1): the pass over the text. For multi-word phrases, at each word position you check n-grams up to the longest phrase length k :

```
for each word position i in chunk:
  for length L from k down to 1: # longest match first
    candidate = normalize(words[i : i+L])
    if candidate in rules: # O(1) hash lookup
      replace, skip ahead L words
  break
```

That's $O(n \cdot k)$ for n words. Since k is small (≤ 4), it's effectively **linear in the chunk length and independent of dictionary size.**

Details to mention:

- **Longest match first**, so "Bajaj Allianz" wins over "Bajaj".
- **Word boundaries**, so "Tata" isn't replaced inside "Tatasky".
- **Case-insensitive** normalization.
- For very large dictionaries, you could use a **trie or Aho-Corasick**: $O(n + \text{matches})$ in a single pass.

Q4. "Zero added latency." What did you measure, and how?

Answer4: "Rules are loaded into memory once, when the bot session starts, not fetched per chunk. A hash-map pass over a ~50-character chunk takes microseconds. TTS time-to-first-byte is 100ms+. I timed the function per chunk in logs, and it was sub-millisecond at p99. Relative to the 800ms budget, it's effectively zero."

Q5: Why not SSML / phonemes / the provider's lexicon?

A phoneme is a basic sound unit of spoken language.

For example, the word "**cat**" is made of roughly three phonemes:

- /k/
- /æ/
- /t/

So instead of telling a TTS engine:

"Say this word normally"

you can tell it the **exact sounds** you want it to produce.

For example, the name "**Bajaj**" might be represented using phonemes to specify exactly how the **j** sounds, where the stress goes, etc.

Think of phonemes as:

"Don't guess the pronunciation. Here are the exact sounds."

This is very precise, but the problem is that **different TTS providers use different phoneme/IPA systems**, and some providers may not support phoneme input at all.

What is SSML?

SSML = Speech Synthesis Markup Language.

It's basically **HTML/XML for speech**.

Instead of sending:

```
Hello, welcome to my application.
```

you can send something like:

```
< speak >
  Hello,
  < break time="500ms" />
  welcome to my application.
< /speak >
```

The TTS engine can then interpret instructions such as:

- pauses
- speaking rate
- pitch
- emphasis
- pronunciation
- different voices
- sometimes phonemes

For example:

```
<prosody rate="slow">
  Welcome to our platform.
</prosody>
```

means roughly:

"Say this more slowly."

1. **Provider-agnostic.** The platform supports multiple TTS providers. SSML and phoneme support varies across them, and many streaming or Indian-language TTS engines barely support it. One layer in front of TTS works with every provider.
2. **DIY users are non-technical.** A builder can write `HDFC` → `एच डी एफ सी` or `Bajaj` → `बजाज`, but they can't write IPA or SSML.
3. **Hindi-English code-mixing.** Respelling in Devanagari is the practical way to fix pronunciation for Indian names.
4. **Per-tenant config in our own database.** Rules are versioned and editable instantly, with no uploading a lexicon to each provider for each voice.

The trade-off (say it before they ask): respelling is less precise than phonemes. Where a provider supports a proper lexicon, that gives more exact control.

Bonus point that impresses interviewers: substitution applies only to the **TTS-bound copy** of the text. The conversation history keeps the original, so the LLM never sees weird respellings, and transcripts and logs stay clean.

Q6: Phrase split across streamed chunks ("Baj" | "aj Allianz")

"The substitution runs **after sentence aggregation**. LLM tokens are buffered into a sentence or clause before going to TTS, so a word is never split mid-token. A phrase crossing a sentence boundary is practically impossible, since brand names don't span punctuation."

It means **you don't run the pronunciation substitution on each small streamed piece of text.**

Suppose the LLM streams:

```
"Baj"  
"aj Allianz"
```

If you process each chunk separately, your system might see:

```
"Baj"      → nothing to replace  
"aj Allianz" → nothing to replace
```

and miss the full phrase **"Bajaj Allianz"**.

Instead, you first **combine the streamed chunks** into a complete sentence/clause:

```
"Baj" + "aj Allianz"  
    ↓  
"Bajaj Allianz"  
    ↓  
apply substitution  
    ↓  
"बजाज Allianz"  
    ↓  
TTS
```

If your layer ran on raw token frames instead

It means: **if you process text immediately as it streams, you temporarily wait when you're unsure whether the current words are part of a larger phrase.**

Example dictionary:

```
"Bajaj Allianz"
```

LLM sends:

```
"Baj"
```

Your system thinks:

"Baj" could be the beginning of "Bajaj Allianz", so don't process it yet.

It **holds "Baj" in a small buffer** and waits for the next chunk:

```
"Baj" + "aj Allianz"
    ↓
"Bajaj Allianz"
    ↓
Now apply substitution
```

What is a prefix set/trie?

It's just a fast lookup structure that answers:

"Could these words be the beginning of one of my known phrases?"

For example:

```
B
Ba
Baj
Baja
Bajaj
Bajaj Allianz
```

If "Baj" is a possible beginning, **hold it**. If it's not, process it immediately.

Why "few milliseconds"?

You only wait when the text **might be part of a known phrase**. Normal text passes through immediately.

Next bullet: Backchannel supervision

Q7. Walk me through the logic. VAD fires while the bot is speaking. What happens next, step by step?

Q8. To know the user said "haan", you need the STT transcript, and that takes 100–200ms. During that window, does the bot keep talking or pause? Why?

Q9. The user says "haan, but wait, mera address galat hai." How does your system avoid swallowing a real interruption that *starts* with a backchannel?

Q10. "Improving turn-taking naturalness." What was the metric? How do you know it improved?

Q11. Why a lexicon and not an ML classifier? Which is worse here, a false positive or a false negative?

Answer 7:

```

on_user_speech_start (while bot is speaking):
start_timer()
# do NOT interrupt yet; bot keeps talking

on_speech_duration_exceeds(400ms):
interrupt_bot() # long speech = real barge-in, no STT wait

on_voicing_stop (before 400ms): # user actually stopped
transcript = normalize(stt_final())
if transcript in BACKCHANNEL_LEXICON:
ignore() # bot continues; NOT added to LLM context
else:
interrupt_bot() # short but meaningful: "ruko", "nahi", "wait"

```

Why each gate exists:

Gate	Job
Duration (~400ms)	Backchannels are short. Anything longer is a real turn, so it interrupts immediately.
Voicing-stop	Don't classify until the user has actually stopped. This prevents judging "haan..." before the rest of the sentence arrives.
Lexicon match	Short does NOT mean ignorable. "Nahi" and "ruko" are short too. Only allowlisted acknowledgments get suppressed.

Point interviewers like: the lexicon is an **allowlist of backchannels**, not a rule that short means ignore. That's the reason the lexicon exists at all.

On "O(1)": here it's legit. The whole short utterance is normalized and looked up as one key in a hash set. For "haan haan" or "hmm okay", you tokenize and check every token, which costs a few lookups.

Normalization matters: STT might output "हाँ", "haan", "han", "hn", or "ha". The lexicon has to cover script and transliteration variants, or it silently fails.

Bonus: a backchannel is never added to the conversation history. If it were, the LLM would try to *respond* to "hmm."

Q8: Does the bot keep talking during the STT wait?

"It keeps talking. Interruption is deferred until the gates decide. The cost is that a real interruption gets delayed, but the duration gate caps that delay: if the user is still speaking at 400ms, I interrupt **without waiting for STT**. So the worst-case delay on a real barge-in is ~400ms, and backchannels never cause a stop."

If they ask what happens when the transcript comes back empty or garbled for a 200ms sound:

"It's under the duration gate with no meaningful words, so treat it as a non-interruption. That also filters coughs and background noise, which is a nice side effect."

Q9: "Haan, but wait, mera address galat hai"

"The voicing-stop gate handles it. I don't classify on hearing 'haan'. I wait for the user to stop. They keep going, so the utterance crosses 400ms and the duration gate interrupts."

If there's a pause after "haan":

"'Haan' ends as its own segment and gets ignored. 'But wait...' starts a **new VAD segment**, which gets evaluated fresh. It's either over 400ms or not in the lexicon, so it interrupts. The cost is a few hundred ms of extra delay in that rare case."

Answer 10: The failure I was fixing is **false interruptions**: the bot cutting itself off on a backchannel. I validated it on test calls with Hindi-English acknowledgments, before and after. The two metrics I'd track in production are the

1. **false-interruption rate** (bot stopped, and the user's utterance was a backchannel)
2. **missed-interruption rate** (the user genuinely interrupted and the bot kept talking). The second one is my guardrail.

Answer 11: **Why a lexicon**:

1. **Deterministic and explainable**. When a call goes wrong, you can see exactly why.
2. **No training data needed**. Labeled Hindi-English backchannel audio barely exists.
3. **Microsecond latency**, compared with running a model inside a real-time loop.
4. **Editable per tenant and language** by a DIY builder.

Trade-off (say it yourself): an ML model using prosody (pitch, intonation) would generalize better, since it could tell a questioning "haan?" from an agreeing "haan." The lexicon is the right v1, and a classifier is the v2 once you have call data to train on.

Which error is worse: a missed real interruption.

"If the bot stops on 'hmm', that's awkward, and it recovers. If the user says 'ruko, galat hai' and the bot keeps talking, the user feels ignored, and in support or collections calls that damages trust. So I biased the design conservatively: **only exact allowlisted backchannels are suppressed; everything else interrupts; long speech always interrupts.**"

Next bullet: RAG test panel

- Q12.** What's a chunk? How were documents chunked, and what chunk size and overlap did you use?
- Q13.** "Relevance score": what is it, exactly? Cosine similarity? What embedding model?
- Q14.** How was the "Would not be used in call" threshold chosen? What happens if it's set too high, and what if it's too low?
- Q15.** Why top-3? What changes if you use top-10?
- Q16.** A builder sees that the right chunk is ranked #5. What can they actually *do* about it using your panel?
- Q17.** Why does latency show up in a *test* panel? Why would a builder care?

Answer 12:

A chunk is a segment of a source document, a few sentences to a paragraph, that gets embedded and indexed as the unit of retrieval. It has to be small enough to be about one thing, so its embedding is precise, and large enough to be self-contained when the LLM reads it.

Chunking strategies:

Strategy	How it works
Fixed-size	N tokens per chunk with overlap, e.g. 400 tokens and ~15% overlap. Overlap keeps a fact that spans a boundary from being split.
Semantic	Embed consecutive sentences and cut where similarity drops, meaning the topic changed. Chunks follow meaning instead of token counts.

Size trade-off (the voice-specific angle scores extra points):

"Small chunks are precise but lose context. Large chunks carry context but dilute the embedding, and they inflate the prompt. In voice, prompt length directly increases LLM time-to-first-token, which eats into the 800ms budget. So voice RAG favors **smaller, tighter chunks and few of them.**"

Hybrid search:

"**Dense** retrieval embeds the query and chunks and compares them by cosine similarity. It catches paraphrases like 'refund' $\approx$ 'paise wapas'. **Sparse** retrieval, BM25, matches keywords, which catches exact terms that embeddings blur: product names, policy numbers, 'EMI', plan codes. I run both and fuse the rankings with **Reciprocal Rank**

Fusion:

$$\text{score}(d) = \sum 1 / (k + \text{rank}_i(d)) , \text{ with } k \approx 60.$$

Hybrid matters here because enterprise knowledge bases are full of names and codes,

and voice queries arrive as **noisy STT transcripts**, where either method alone misses things."

Answer 13:

"It was the reranker score. Hybrid search (dense + BM25, fused with RRF) retrieves a candidate set, the reranker scores each candidate against the query, and the panel shows the top 3 by reranker score. The red threshold applies to that score."

What a reranker is. They will ask:

	Bi-encoder (embeddings)	Cross-encoder (reranker)
How it works	Query and chunk are embedded separately , then compared with cosine similarity	Query and chunk go through the model together , so every query token attends to every chunk token
Speed	Fast. Chunk embeddings are precomputed, so search is a vector lookup	Slow. One model call per (query, chunk) pair
Precision	Coarse	High. It directly outputs "how relevant is this chunk to this query"
Use	Retrieval: narrow millions down to ~20	Reranking: pick the best 3 out of ~20

"You can't rerank the whole corpus, because that's one model call per chunk. So retrieval narrows it down cheaply, and the reranker does the expensive, precise pass on the shortlist."

Why the reranker score is the right thing to threshold on:

"RRF is rank-based, so it only orders results. Cosine similarity is a proxy. The reranker score is a direct query-chunk relevance judgment, so a fixed cut-off actually means something. It's still **model-specific**, though: change the reranker model and you have to re-tune the threshold."

Answer 14:

It was tuned on the **reranker score**. I ran test queries where the correct chunk was known, compared the reranker scores of correct vs incorrect chunks, and set the cut-off where they separated. It's a precision/recall trade-off. Because the score comes from a specific model, the threshold is **tied to that reranker**: swap the model and you re-tune it.

Threshold	What happens on the call
Too high	Correct chunks get dropped. The bot says "I don't know", or answers from its own training data and hallucinates .

Threshold	What happens on the call
Too low	Weak chunks enter the prompt. The LLM gives confident wrong answers , which is dangerous in BFSI, and the extra tokens add latency.

"The pipeline is **hybrid retrieve N → rerank → top 3 into the prompt**. The reranker already handles precision, so 3 high-confidence chunks is enough. Going to 10 costs three things:

1. **Latency:** more prompt tokens means slower time-to-first-token inside the 800ms budget
2. **Quality:** lower-ranked chunks distract the LLM
3. **Fit:** voice answers are one or two facts, not essays

The panel shows 3 because runtime uses 3."

Follow-up: "What if the right chunk isn't in the top N from retrieval?"

"Then the reranker can't save it. **Recall is decided at retrieval; precision at reranking**. The fix is raising N, which trades against reranker latency, or improving the knowledge base content."

Answer15:

"The pipeline is **hybrid retrieve N → rerank → top 3 into the prompt**. The reranker already handles precision, so 3 high-confidence chunks is enough. Going to 10 costs three things:

1. **Latency:** more prompt tokens means slower time-to-first-token inside the 800ms budget
2. **Quality:** lower-ranked chunks distract the LLM
3. **Fit:** voice answers are one or two facts, not essays

The panel shows 3 because runtime uses 3."

Follow-up: "What if the right chunk isn't in the top N from retrieval?"

"Then the reranker can't save it. **Recall is decided at retrieval; precision at reranking**. The fix is raising N, which trades against reranker latency, or improving the knowledge base content."

Answer 16:

The panel **diagnoses** the problem. The builder fixes it in the knowledge base:

1. **Split or rewrite the source doc.** If the chunk covers several topics, the reranker scores it lower.

2. **Add customers' actual vocabulary:** synonyms and Hinglish phrasing ("paise wapas" next to "refund").
3. **Add an FAQ-style entry** that mirrors the question.
4. **Remove duplicate or conflicting docs** that outrank it.
5. **Re-run the query in the panel** to confirm it's now in the top 3 and above the threshold.

Seeing it at #5 also tells the builder something useful:

"It was **retrieved** but **reranked low**. That means recall is fine and the problem is the chunk's content. If it didn't show up at all, it would be a retrieval miss."

Answer 17:

"Retrieval sits **inside the 800ms budget**, and the reranker is usually its slowest step, anywhere from tens to 100+ ms depending on N and whether it's an API or self-hosted. Slow retrieval means dead air on a live call. Showing latency per query lets a builder see the cost of their setup **before going live**: a huge knowledge base, a big N, or long chunks all show up here instead of in customer complaints."

Breakdown to say out loud:

"Query embedding call → vector + BM25 search → fusion → rerank. Embedding and reranking are the network- and model-heavy steps. Search itself is milliseconds."

Follow-up 1: "What's N? What if the right chunk isn't in the top N?"

What's N:

"N is how many first-stage candidates go to the reranker. For voice it's kept small, around 10–20, because a cross-encoder scores every (query, chunk) pair, so reranker cost grows with N."

⚠️ Know your actual N. If you don't, say *"around 20, from what I recall"* and move straight into how you'd *choose* it (below). That turns a gap into a strength.

If the right chunk isn't in the top N:

"The reranker only reorders what it's given, so it can't recover a chunk that retrieval missed. **Recall is decided at retrieval; precision at reranking.** The fixes are on the first stage:

1. **Raise N.** Recall goes up, but so does reranker latency.
2. **Improve first-stage recall.** Hybrid search already helps. Beyond that: a better multilingual, Hinglish-aware embedding model, and better chunking.
3. **Query rewriting.** STT transcripts are noisy, so normalize or rewrite the query before retrieval. That costs latency, so use it only where it pays off.

4. **Fix the knowledge base content**, which is what the test panel helps builders do."

Chained follow-up: "How do you pick N?"

"Take a labeled set of queries with known correct chunks and measure **recall@N**: the fraction of queries whose correct chunk appears in the top N. Plot it against N. Recall rises and then plateaus. Pick N at the knee, then check that the reranker latency at that N fits the budget."

Chained follow-up: "How would you evaluate retrieval in general?"

Metric	Meaning
Recall@k	Is the correct chunk anywhere in the top k? (retrieval stage)
Precision@k	What fraction of the top k is relevant?
MRR	Mean of 1/rank of the first correct chunk. Rewards putting it at #1.
nDCG	Rank-weighted relevance when several chunks are relevant to different degrees

"For this pipeline: **recall@N for retrieval, hit@3 or MRR after reranking.**"

Follow-up 2: "Could you skip the reranker to save latency?"

When it's fine:

"For small, clean FAQ knowledge bases where first-stage top-3 is already correct. You don't guess this, you **measure** it: run the labeled query set with and without the reranker and compare hit@3. If the gain is negligible, drop it for that bot."

The hidden cost that most candidates miss. Say this:

"Without the reranker, you lose the **thresholdable score**. RRF is rank-based, and cosine similarity is a weak proxy. So skipping the reranker also weakens the 'would not be used in call' filter. You'd have to fall back to a cosine threshold tuned per embedding model, which is less reliable."

Better options than skipping it entirely:

1. **Conditional reranking.** If the first-stage top-1 is far ahead of #2, you're already confident, so skip. Rerank only when the results are close.
2. **Lower N.** Most of the gain usually comes from the top candidates anyway.
3. **Smaller or self-hosted reranker**, co-located with the pipeline to cut network time.
4. **Cache frequent queries.** FAQ-style bots get the same questions again and again, so cache the query embedding and results.
5. **Start retrieval early** on the partial STT transcript, so it overlaps with endpointing.

Last internship bullet: STT language auto-detection bug

Q18. What was the *symptom*? How did you first notice it, and how did you reproduce it?

Q19. "Auto-detection enabled but detector not activated." Explain that. Where was the gap: a config flag, the API request params, or something else?

Q20. Why does the detected language need to reach the **system prompt**? What broke when it didn't?

Q21. A user speaks Hinglish. Which language gets "detected"? Is detection per utterance or per call? What if they switch mid-call?

Q22. How did you verify the fix, and how would you stop it from regressing?

Answer 18:

Hindi-speaking callers were getting transcripts in the **wrong language**: Hindi audio came back as phonetically mangled English words, so the bot replied to nonsense. It showed up in call transcripts and logs. To reproduce it, I took a fixed set of Hindi test audio clips, ran them through the pipeline with auto-detect 'enabled', and inspected the **raw request and response to the STT provider**, not just the final transcript.

Answer 19:

"The setting was on in the bot's config, but it never reached the STT API request. The request still went out with a **fixed default language**, so the provider transcribed everything as that language.

I debugged it by **tracing the flag end to end**:

UI toggle → stored bot config → pipeline init → STT service constructor → **outbound request payload**.

Logging the actual payload showed the language parameter was hard-set, so auto-detection was never requested."

Answer 20:

"STT detection tells you *what language the user is speaking*. The LLM needs that to **reply in the same language**, and TTS needs it to pick a matching voice. Detection returned the language as metadata on the transcript, but it was never propagated into the prompt. So the user spoke Hindi, and the LLM replied in the prompt's default language, English. Or TTS read Hindi text with an English voice and it came out garbled.

The fix was to inject the detected language into the system prompt ('Respond in {language}') and keep TTS in sync with it."

So it was two bugs, and the bullet mentions both:

1. Detection wasn't requested → wrong transcript.
2. The detection result wasn't propagated → wrong reply language.

On-prem model deployment

On-prem = on your own servers/infrastructure, instead of sending data to a cloud provider.

- **T1.** Why would an enterprise want on-prem? What changes for latency and cost compared with cloud APIs?
- **T2.** Which models ran on-prem (STT/LLM/TTS)? What hardware?

Answer T1:

Why enterprises want it:

1. **Compliance and data residency.** BFSI call audio carries PII: names, account numbers, loan details. Regulators and the DPDP Act push toward data not leaving their network.
2. **Security review.** Many banks simply won't approve sending call audio to third-party APIs.
3. **Predictable cost at high volume.** Per-minute API pricing adds up with thousands of calls a day. Owning the GPUs is a fixed cost.

"Cloud STT, LLM, and TTS are three separate network round trips, each 50–150ms, across different providers. On-prem you **co-locate all three** on the same network or GPU box, so those hops disappear. The catch is that inference speed now depends on your own GPUs."

Trade-offs (say them yourself):

- GPU capex and an ops burden: monitoring, upgrades, failures are all yours
- **Limited to open models**, which are often smaller than frontier APIs, so quality can drop
- **No elastic scaling.** Capacity is fixed, so traffic spikes need admission control.

Answer T2: Answer the stack here.

2. Pipecat (it's in your skills section):

- **T3.** How does Pipecat work internally? Frames, processors, pipeline. Where would your pronunciation layer and backchannel logic plug in?
- **T4.** How does an interruption travel through a Pipecat pipeline?

Answer T3:

"A Pipecat **pipeline** is a chain of **frame processors**. Everything moving through it is a **frame**: audio frames, transcription frames, LLM text frames, and control frames like start, end, and interruption. Each processor implements `process_frame(frame, direction)`, does its job, and calls `push_frame()` to pass the frame on. Frames flow downstream and can also go upstream."

Typical pipeline:

```

transport.input()          # audio in + VAD analyzer (e.g., Silero)
  → STT service            # emits TranscriptionFrames
  → user context aggregator
  → LLM service            # emits text frames (streamed)
  → TTS service            # emits audio frames
  → transport.output()     # audio out
  → assistant context aggregator

```

Where the pronunciation layer fits, and a subtlety that impresses interviewers:

"The obvious place is a custom processor between the LLM and TTS that rewrites text frames. But the assistant context aggregator downstream would then record the *respelled* text into the conversation history. To keep the history clean, the substitution belongs **inside the TTS call itself**, for example by subclassing the TTS service and substituting just before synthesis. Then only the audio gets the respelled text, and the frames flowing downstream stay original."

Backchannel:

"It sits at the interruption decision. Pipecat lets you customize when a user's speech is allowed to interrupt the bot, including pluggable interruption strategies (a minimum-words strategy, for example). My logic, meaning the duration gate plus lexicon plus voicing-stop, decides whether an interruption goes through or gets swallowed."

Answer T4:

- "1. VAD in the input transport detects the user started speaking.
2. If interruptions are allowed and the strategy approves, an **interruption frame** is pushed downstream.
3. It's a **system frame**, so it **skips the normal queues** and propagates immediately, because interruptions can't wait behind buffered audio.
4. Each processor reacts as it passes: the **LLM** cancels generation, **TTS** stops and drops pending text and audio, and the **output transport** flushes its audio buffer, so playback stops.
5. The assistant context records only what was **actually spoken**, so the LLM knows what the user heard.
6. The user's new speech then flows through STT as a normal turn."

3. A system design extension. This is common for voice AI internships:

- **T5.** Scale this voicebot to 1,000 concurrent calls. What's the bottleneck, and how do you scale it?
- **T6.** A call drops mid-conversation. How do you debug it?

Answer T5:

Telephony / SIP gateway

- Load balancer (routes new calls to the least-loaded worker)
- Pipeline workers (one pipeline per call, async; autoscaled on ACTIVE CALL COUNT, not CPU)
- STT / LLM / TTS model servers (shared pools, batched)
- Vector DB + reranker
- Redis (bot config cache, session state)
- Postgres (bot configs, call records)
- Queue → post-call workers (transcripts, analytics, webhooks, recordings to object storage)

Bottlenecks, and how to handle each:

Bottleneck	Fix
Model inference (usually #1)	On-prem: GPU pools with continuous batching (vLLM for the LLM), sized for peak concurrent streams. Cloud: provider concurrency and rate limits , so negotiate limits and keep a fallback provider.
Worker CPU (VAD, resampling, audio I/O)	Horizontal pods; measure calls per worker and scale on that number
Tail latency under load	Monitor p99 per stage. The average hides the dead-air calls.
Capacity full	Admission control : queue or reject new calls gracefully instead of degrading every live call
Deploys	Graceful drain : stop routing new calls to old pods and let active calls finish. You can't kill live calls.

Answer T6:

First, clarify:

"Is it a real **disconnect**, or **dead air** where the call is up but the bot went silent? They're different bugs."

Then trace it with the call ID across every layer:

Layer	What to check
Telephony	Who sent the hang-up (user, carrier, or us)? SIP error codes, network jitter and packet loss
Pipeline	Exceptions, a processor crash, an unexpected end/cancel frame, an idle timeout (the bot hung up because it thought the user was silent)
Providers	STT/TTS websocket disconnects, timeouts, 429 rate limits

Layer	What to check
Infra	Worker OOM, pod restart, a deploy mid-call

Dead air specifically: the LLM or TTS hung and never returned.

Prevention:

- **Per-stage timeouts** with a fallback line ("ek second, main check karta hoon")
- **Reconnect and retry** for provider websockets; a **fallback provider**
- **Structured logs with a call/trace ID** across every service
- **Alert on drop rate** by cause

Hardest problem (use the STT bug; debugging stories score best)

"Situation: Hindi callers were getting wrong-language transcripts even though auto-detect was enabled for the bot.

What made it hard: everything *looked* correct. The config said enabled, and there was no error anywhere.

Approach: I stopped trusting the config and traced the setting end to end, from the bot config through pipeline init to the actual request sent to the STT provider.

Root cause: there were two bugs. The detector was never actually requested in the payload, and even once detection worked, the detected language wasn't reaching the system prompt, so replies came back in the wrong language.

Fix and lesson: I fixed both, then verified with replayed Hindi, English, and Hinglish audio. The lesson I took away: verify at the boundary, meaning the request that actually leaves your system, not the config that says what should happen."

I2: Biggest contribution

Pick based on the role:

For SDE / product roles, the RAG test panel:

"The RAG test panel, because it was the one I owned end to end: the retrieval call, reranker scores, threshold logic, and the UI. It also solved a real product gap. Builders had no way to see *why* their bot answered wrong until a live call failed. The panel moved that discovery from production to before go-live."

For AI / voice roles, backchannel supervision:

"Backchannel supervision, because it's a real-time problem where every millisecond matters, and it directly affects how human the bot feels. The design choice I'm proudest of is treating the lexicon as an allowlist: short utterances aren't automatically ignored, only known acknowledgments are, so 'ruko' still interrupts."

I4: How work got reviewed and shipped

Pick the one that's true:

- **With PRs and review:** "I raised PRs reviewed by [senior/lead], tested on [staging / test calls], then merged."
- **Informal:** "It was a small team, so reviews were quick demos and walkthroughs with [lead] before merging, and I tested with scripted test calls."

Add this either way, since it's true of everything you've described:

"For the voice features, I validated on test calls covering Hindi, English, and Hinglish before merging, because unit tests alone don't catch real-time behavior."

I5: How you worked with the team

Reframe the PRD part, which you mentioned passively in Q1:

"It was a 5-person team. I started by going through the product PRDs, which meant I understood *why* each feature existed before writing code. That's a big part of why my work leaned toward product problems like call quality and builder tooling. Tasks came from [lead/founder], and the others owned [the core pipeline / infra / frontend / customer deployments]. My work sat between the product and the pipeline."

I6: Production impact

Pick the one that's true:

- **Went live:** "[Feature] went live on customer bots. The effect was [fewer false interruptions / builders fixing retrieval before launch]."
- **Not live yet / internal:** "The panel was used internally by the team and builders during bot setup. The runtime changes were validated on test calls. I don't have production metrics, and if I'd stayed longer, measuring that impact is the first thing I'd have done."

The second version is honest and still strong. Admitting there's no metric *and* saying how you'd measure it beats a vague claim.

I7: One more month (opinion, so this one's fully yours to use)

"Three things, in order:

1. **Measure what I built.** False-interruption rate for backchannels, and retrieval hit rate on real call transcripts.
2. **Retrieval regression tests in the panel.** Saved test queries that re-run whenever a builder changes the knowledge base.
3. **A prosody-based backchannel classifier as v2**, trained on real call audio, so it can tell a questioning 'haan?' from an agreeing 'haan.'"